

JavaScript & Programming Foundations Lab

Identificação

Lab: JavaScript & Programming Foundations Lab

Sigla: LJS

Finalidade: Construir uma base que permita **analisar problemas, formular algoritmos, prever a execução de programas e expressar soluções corretamente em JavaScript**, compreendendo os mecanismos da linguagem em vez de apenas memorizar sintaxe.

Visão geral: O **JavaScript & Programming Foundations Lab** integra raciocínio algorítmico aos mecanismos fundamentais do JavaScript: parte da representação de problemas, valores, estado e execução; evolui para controle de fluxo, funções, modelagem e processamento de dados; depois consolida o modelo de objetos, modularização, erros e assincronismo. A organização não replica a estrutura da especificação, do MDN ou da CS2023: ela resulta da combinação entre fundamentos introdutórios de programação e dependências reais dos mecanismos da linguagem.

Escopo: O Lab cobre o núcleo de lógica de programação e ECMAScript necessário para formação inicial profissional, incluindo um modelo mínimo de assincronismo. APIs do navegador, DOM, Node.js, frameworks, estruturas de dados especializadas e análise algorítmica formal ficam fora do núcleo; ECMAScript é tratado como linguagem executada dentro de um host, e não confundido com as APIs desse ambiente.

Estrutura curricular

LJS-01 — Problemas, valores, estado e execução

Objetivo

Construir o primeiro modelo mental de programa: representar um problema como dados e operações cuja execução transforma estado de maneira previsível.

Unidades

LJS-01.01 — Problemas e algoritmos **Objetivo:** transformar problemas simples em processos computacionais explícitos antes de se concentrar na sintaxe.

Conceitos fundamentais:

1. problema computacional, objetivo e resultado esperado;
2. entradas, saídas, regras e restrições;
3. decomposição de problemas;
4. abstração e identificação das informações relevantes;
5. algoritmo como sequência finita de operações;
6. estado inicial, transformações e estado resultante;
7. rastreamento passo a passo e casos-limite.

Chats derivados:

- LJS-01.01 - Estudo - Problemas e algoritmos
- LJS-01.01 - Prática - Problemas e algoritmos

LJS-01.02 — Valores, tipos e bindings **Objetivo:** compreender como JavaScript representa valores e associa nomes a eles.

Conceitos fundamentais:

1. valor, tipo e tipagem dinâmica;
2. valores primitivos e objetos;
3. `undefined`, `null`, booleanos, strings e números;
4. particularidades fundamentais de `Number`: `NaN` e infinitos;

5. identificadores e bindings;
6. declaração e inicialização;
7. `let` e `const`;
8. reatribuição versus mutação.

Chats derivados:

- LJS-01.02 - Estudo - Valores, tipos e bindings
- LJS-01.02 - Prática - Valores, tipos e bindings

LJS-01.03 — Expressões e transformação de valores **Objetivo:** raciocinar sobre como expressões produzem resultados e transformam dados.

Conceitos fundamentais:

1. expressões, operandos e resultados;
2. avaliação, precedência e associatividade;
3. operações aritméticas, relacionais e lógicas no contexto de problemas;
4. conversão explícita e coerção implícita;
5. igualdade estrita e igualdade abstrata;
6. `truthy` e `falsy`;
7. curto-circuito com `&&` e `||`;
8. ausência de valor e `??`;
9. expressão condicional.

Chats derivados:

- LJS-01.03 - Estudo - Expressões e transformação de valores
- LJS-01.03 - Prática - Expressões e transformação de valores

Resultado esperado

Conseguir decompor problemas simples, representar seus dados como valores e bindings e prever como expressões e atribuições alteram o estado de um programa.

Checkpoint derivado

LJS-01 - Checkpoint - Problemas, valores, estado e execução

LJS-02 — Decisão, repetição e fluxo de execução

Objetivo

Transformar algoritmos lineares em algoritmos capazes de selecionar caminhos e repetir processamento, mantendo explícito o raciocínio sobre estado, progresso e término.

Unidades

LJS-02.01 — Decisão e caminhos de execução **Objetivo:** modelar alternativas de execução a partir de condições.

Conceitos fundamentais:

1. condição e predicado;
2. estado antes e depois de uma decisão;
3. caminhos mutuamente exclusivos e caminhos opcionais;
4. `if`, `else if` e `else`;
5. seleção por casos com `switch`;
6. condições compostas;
7. decisões aninhadas;

8. cobertura de casos e casos-limite.

Chats derivados:

- LJS-02.01 - Estudo - Decisão e caminhos de execução
- LJS-02.01 - Prática - Decisão e caminhos de execução

LJS-02.02 — Repetição e progresso **Objetivo:** representar processos repetitivos garantindo que cada iteração contribua para o objetivo.

Conceitos fundamentais:

1. repetição baseada em condição;
2. repetição baseada em contagem;
3. `while`;
4. `for`;
5. iteração sobre sequências com `for...of`;
6. estado de iteração;
7. inicialização, progresso e condição de término;
8. `break` e `continue`;
9. loops infinitos e condições de parada.

Chats derivados:

- LJS-02.02 - Estudo - Repetição e progresso
- LJS-02.02 - Prática - Repetição e progresso

LJS-02.03 — Rastreamento e correção intuitiva **Objetivo:** desenvolver capacidade de explicar por que um algoritmo termina e produz determinado resultado.

Conceitos fundamentais:

1. rastreamento de variáveis e caminhos;
2. invariantes intuitivos;
3. pré-condições e resultados esperados;
4. casos normais, vazios e extremos;
5. término;
6. erros lógicos;
7. validação manual de pequenas soluções.

Chats derivados:

- LJS-02.03 - Estudo - Rastreamento e correção intuitiva
- LJS-02.03 - Prática - Rastreamento e correção intuitiva

Resultado esperado

Conseguir prever e construir fluxos com decisões e repetições, identificando como cada caminho e iteração afeta o estado e o resultado.

Checkpoint derivado

LJS-02 - Checkpoint - Decisão, repetição e fluxo de execução

LJS-03 — Funções, escopo e abstração

Objetivo

Usar funções como principal mecanismo de decomposição e abstração de comportamento, compreendendo também como escopo e chamadas determinam a execução.

Unidades

LJS-03.01 — Decomposição funcional **Objetivo:** dividir soluções em operações menores com responsabilidades explícitas.

Conceitos fundamentais:

1. função como unidade de comportamento;
2. responsabilidade e contrato conceitual;
3. parâmetros e argumentos;
4. processamento e retorno;
5. funções auxiliares;
6. decomposição de algoritmos;
7. composição de resultados;
8. produção de valores versus efeitos observáveis.

Chats derivados:

- LJS-03.01 - Estudo - Decomposição funcional
- LJS-03.01 - Prática - Decomposição funcional

LJS-03.02 — Funções como valores **Objetivo:** compreender a característica de primeira classe das funções em JavaScript.

Conceitos fundamentais:

1. função como valor;
2. referências a funções;
3. function declarations e function expressions;
4. arrow functions;
5. callbacks;
6. funções recebendo funções;
7. funções retornando funções;
8. funções de ordem superior.

Chats derivados:

- LJS-03.02 - Estudo - Funções como valores
- LJS-03.02 - Prática - Funções como valores

LJS-03.03 — Escopo, bindings e closures **Objetivo:** compreender como nomes são resolvidos e como funções preservam acesso ao ambiente em que foram criadas.

Conceitos fundamentais:

1. escopo léxico;
2. escopo de bloco;
3. escopo de função;
4. resolução de identificadores;
5. shadowing;
6. temporal dead zone;
7. ambientes léxicos;
8. closures;
9. estado capturado por closures.

Chats derivados:

- LJS-03.03 - Estudo - Escopo, bindings e closures
- LJS-03.03 - Prática - Escopo, bindings e closures

LJS-03.04 — Chamadas e execução síncrona **Objetivo:** relacionar chamadas de função com a ordem efetiva de execução.

Conceitos fundamentais:

1. contexto de execução;
2. chamada, entrada e retorno de função;
3. call stack;
4. funções aninhadas e cadeia de chamadas;
5. propagação de resultados;
6. execução síncrona;
7. execução até a conclusão de cada trabalho iniciado.

Chats derivados:

- LJS-03.04 - Estudo - Chamadas e execução síncrona
- LJS-03.04 - Prática - Chamadas e execução síncrona

Resultado esperado

Conseguir decompor programas em funções, manipular funções como valores e explicar corretamente escopo, closures e a sequência de chamadas.

Checkpoint derivado

LJS-03 - Checkpoint - Funções, escopo e abstração

LJS-04 — Modelagem de dados, identidade e mutabilidade

Objetivo

Representar entidades e coleções com estruturas adequadas e compreender as consequências de compartilhar e modificar objetos.

Unidades

LJS-04.01 — Sequências e registros **Objetivo:** escolher representações simples para dados estruturados.

Conceitos fundamentais:

1. strings como valores textuais;
2. arrays como sequências ordenadas;
3. índices e comprimento;
4. objetos como conjuntos de propriedades;
5. chaves e valores;
6. acesso e atualização de propriedades;
7. estruturas aninhadas;
8. arrays de objetos e objetos contendo arrays;
9. destructuring e spread como sintaxes auxiliares de manipulação.

Chats derivados:

- LJS-04.01 - Estudo - Sequências e registros
- LJS-04.01 - Prática - Sequências e registros

LJS-04.02 — Identidade, compartilhamento e cópia **Objetivo:** evitar o modelo incorreto de que objetos simplesmente são “passados por referência”.

Conceitos fundamentais:

1. valores primitivos versus identidade de objetos;

2. igualdade e identidade;
3. bindings contendo valores de objetos;
4. mutação;
5. aliasing e compartilhamento;
6. argumentos passados por valor e compartilhamento de objetos;
7. reatribuição versus alteração do objeto;
8. cópia superficial;
9. efeitos de estruturas aninhadas compartilhadas.

Chats derivados:

- LJS-04.02 - Estudo - Identidade, compartilhamento e cópia
- LJS-04.02 - Prática - Identidade, compartilhamento e cópia

Resultado esperado

Conseguir modelar dados com arrays e objetos e prever corretamente os efeitos de identidade, mutação, compartilhamento e cópias.

Checkpoint derivado

LJS-04 - Checkpoint - Modelagem de dados, identidade e mutabilidade

LJS-05 — Processamento de coleções e raciocínio algorítmico

Objetivo

Reconhecer padrões recorrentes de processamento de dados independentemente do método específico usado para implementá-los.

Unidades

LJS-05.01 — Padrões de processamento **Objetivo:** reconhecer a intenção algorítmica por trás de diferentes operações sobre coleções.

Conceitos fundamentais:

1. percurso;
2. transformação;
3. seleção;
4. busca;
5. teste existencial e universal;
6. agregação;
7. acumulação de estado;
8. encerramento antecipado;
9. combinação de padrões.

Chats derivados:

- LJS-05.01 - Estudo - Padrões de processamento
- LJS-05.01 - Prática - Padrões de processamento

LJS-05.02 — Expressando padrões em JavaScript **Objetivo:** relacionar os padrões anteriores às construções adequadas da linguagem.

Conceitos fundamentais:

1. loops como forma geral de processamento;
2. callbacks sobre coleções;
3. `map()` como transformação;

4. `filter()` como seleção;
5. `find()` como busca;
6. `some()` e `every()` como testes;
7. `reduce()` como agregação;
8. escolha entre método iterativo e loop explícito.

Chats derivados:

- LJS-05.02 - Estudo - Expressando padrões em JavaScript
- LJS-05.02 - Prática - Expressando padrões em JavaScript

LJS-05.03 — Ordenação e custo intuitivo **Objetivo:** desenvolver sensibilidade inicial ao custo das escolhas algorítmicas sem antecipar análise assintótica formal.

Conceitos fundamentais:

1. ordenação como transformação de uma coleção;
2. função comparadora;
3. percursos completos e encerramento antecipado;
4. percursos repetidos;
5. loops aninhados;
6. crescimento intuitivo do trabalho com a entrada;
7. comparação de soluções equivalentes;
8. adequação entre solução, clareza e custo.

Chats derivados:

- LJS-05.03 - Estudo - Ordenação e custo intuitivo
- LJS-05.03 - Prática - Ordenação e custo intuitivo

Resultado esperado

Conseguir resolver problemas sobre coleções por padrões de transformação, seleção, busca, teste, agregação e ordenação, avaliando intuitivamente o trabalho necessário.

Checkpoint derivado

LJS-05 - Checkpoint - Processamento de coleções e raciocínio algorítmico

LJS-06 — Objetos, protótipos e classes

Objetivo

Construir o modelo de objetos específico do JavaScript antes de introduzir classes, evitando tratá-las como um sistema independente de orientação a objetos.

Unidades

LJS-06.01 — Propriedades e cadeia de protótipos **Objetivo:** compreender como propriedades são localizadas e comportamentos podem ser compartilhados.

Conceitos fundamentais:

1. propriedades próprias;
2. propriedades herdadas;
3. `[[Prototype]]` como modelo conceitual;
4. prototype chain;
5. resolução de propriedades;
6. compartilhamento de comportamento;
7. métodos como propriedades cujo valor é função.

Chats derivados:

- LJS-06.01 - Estudo - Propriedades e cadeia de protótipos
- LJS-06.01 - Prática - Propriedades e cadeia de protótipos

LJS-06.02 — Métodos, `this` e construção **Objetivo:** compreender como a forma de chamada influencia o contexto de uma função.

Conceitos fundamentais:

1. chamada de função versus chamada de método;
2. determinação de `this` em funções comuns;
3. `this` como parte do contexto de chamada;
4. arrow functions e `this` léxico;
5. funções construtoras como conceito de suporte;
6. operador `new`;
7. instância e protótipo compartilhado.

Chats derivados:

- LJS-06.02 - Estudo - Métodos, `this` e construção
- LJS-06.02 - Prática - Métodos, `this` e construção

LJS-06.03 — Classes sobre o modelo prototípico **Objetivo:** usar a sintaxe de classes mantendo explícita sua relação com o sistema de objetos da linguagem.

Conceitos fundamentais:

1. classe como mecanismo de definição de objetos;
2. constructor;
3. criação de instâncias;
4. métodos de instância;
5. relação entre classe, instância e protótipo;
6. `this` em métodos e construtores.

Chats derivados:

- LJS-06.03 - Estudo - Classes sobre o modelo prototípico
- LJS-06.03 - Prática - Classes sobre o modelo prototípico

Resultado esperado

Conseguir explicar propriedades próprias e herdadas, prototype chain, `this`, `new` e classes como partes relacionadas do mesmo modelo de objetos.

Checkpoint derivado

LJS-06 - Checkpoint - Objetos, protótipos e classes

LJS-07 — Módulos e falhas

Objetivo

Passar de programas monolíticos para unidades com dependências explícitas e desenvolver um modelo fundamental de falhas e propagação.

Unidades

LJS-07.01 — Módulos ECMAScript **Objetivo:** organizar código por responsabilidades sem confundir sintaxe da linguagem com carregamento do ambiente.

Conceitos fundamentais:

1. módulo como unidade de código;
2. escopo de módulo;
3. dependências;
4. `export`;
5. `import`;
6. exports nomeados;
7. `export default` e sua distinção conceitual;
8. bindings importados;
9. grafo básico de módulos.

Chats derivados:

- LJS-07.01 - Estudo - Módulos ECMAScript
- LJS-07.01 - Prática - Módulos ECMAScript

LJS-07.02 — Erros e exceções **Objetivo:** distinguir diferentes classes de falha e controlar erros recuperáveis.

Conceitos fundamentais:

1. erros de sintaxe;
2. erros em execução;
3. erros lógicos;
4. objetos `Error`;
5. `throw`;
6. `try`;
7. `catch`;
8. `finally`;
9. propagação por chamadas;
10. recuperação versus propagação intencional.

Chats derivados:

- LJS-07.02 - Estudo - Erros e exceções
- LJS-07.02 - Prática - Erros e exceções

Resultado esperado

Conseguir estruturar código em módulos ECMAScript e explicar como erros surgem, propagam-se e podem ser tratados.

Checkpoint derivado

LJS-07 - Checkpoint - Módulos e falhas

LJS-08 — Execução assíncrona e ambientes JavaScript

Objetivo

Fornecer o modelo mínimo de assincronismo necessário para trabalhar posteriormente com browser e Node.js sem atribuir ao ECMAScript mecanismos pertencentes ao host.

Unidades

LJS-08.01 — Linguagem, engine, runtime e host **Objetivo:** estabelecer corretamente as responsabilidades das diferentes camadas.

Conceitos fundamentais:

1. ECMAScript como especificação da linguagem;

2. engine como implementação da linguagem;
3. runtime e ambiente hospedeiro;
4. APIs fornecidas pelo host;
5. execução síncrona e call stack;
6. operações externas ao fluxo síncrono;
7. jobs ECMAScript;
8. event loop como mecanismo do ambiente;
9. diferença conceitual entre browser e Node.js.

Chats derivados:

- LJS-08.01 - Estudo - Linguagem, engine, runtime e host
- LJS-08.01 - Prática - Linguagem, engine, runtime e host

LJS-08.02 — Promises **Objetivo:** representar e compor resultados que podem ficar disponíveis posteriormente.

Conceitos fundamentais:

1. operação futura e resultado eventual;
2. Promise;
3. pending, fulfilled e rejected;
4. resolução e rejeição;
5. `then()`;
6. encadeamento e retorno de Promises;
7. `catch()`;
8. `finally()`;
9. propagação de valores e erros;
10. coordenação básica de operações independentes com `Promise.all()`.

Chats derivados:

- LJS-08.02 - Estudo - Promises
- LJS-08.02 - Prática - Promises

LJS-08.03 — async e await **Objetivo:** utilizar sintaxe assíncrona a partir do modelo já construído de Promises.

Conceitos fundamentais:

1. funções `async` como funções orientadas a Promises;
2. `await` sobre resultados assíncronos;
3. suspensão da continuação da função;
4. retomada posterior;
5. rejeição convertida em fluxo de erro;
6. `try...catch` com `await`;
7. dependências sequenciais;
8. operações independentes;
9. diferença entre sequência lógica e concorrência assíncrona.

Chats derivados:

- LJS-08.03 - Estudo - async e await
- LJS-08.03 - Prática - async e await

LJS-08.04 — Jobs, tasks e microtasks **Objetivo:** conhecer apenas a camada necessária para prever a ordem básica do código assíncrono.

Conceitos fundamentais:

1. trabalho síncrono até a conclusão;
2. Promise reactions como jobs;
3. microtasks no modelo do host Web;
4. tasks versus microtasks no navegador;

5. checkpoints de microtasks;
6. relação conceitual entre Promise e fila de microtasks;
7. diferenças de runtime;
8. timers como APIs do host, não da linguagem.

Chats derivados:

- LJS-08.04 - Estudo - Jobs, tasks e microtasks
- LJS-08.04 - Prática - Jobs, tasks e microtasks

Resultado esperado

Conseguir explicar Promises e `async/await`, prever a ordem básica de execução assíncrona e distinguir claramente o que pertence ao ECMAScript do que pertence ao browser ou ao Node.js.

Checkpoint derivado

LJS-08 - Checkpoint - Execução assíncrona e ambientes JavaScript

Mapa operacional dos chats

LJS-01 - Problemas, valores, estado e execução

LJS-01.01 - Estudo - Problemas e algoritmos

LJS-01.01 - Prática - Problemas e algoritmos

LJS-01.02 - Estudo - Valores, tipos e bindings

LJS-01.02 - Prática - Valores, tipos e bindings

LJS-01.03 - Estudo - Expressões e transformação de valores

LJS-01.03 - Prática - Expressões e transformação de valores

LJS-01 - Checkpoint - Problemas, valores, estado e execução

LJS-02 - Decisão, repetição e fluxo de execução

LJS-02.01 - Estudo - Decisão e caminhos de execução

LJS-02.01 - Prática - Decisão e caminhos de execução

LJS-02.02 - Estudo - Repetição e progresso

LJS-02.02 - Prática - Repetição e progresso

LJS-02.03 - Estudo - Rastreamento e correção intuitiva

LJS-02.03 - Prática - Rastreamento e correção intuitiva

LJS-02 - Checkpoint - Decisão, repetição e fluxo de execução

LJS-03 - Funções, escopo e abstração

LJS-03.01 - Estudo - Decomposição funcional

LJS-03.01 - Prática - Decomposição funcional

LJS-03.02 - Estudo - Funções como valores

LJS-03.02 - Prática - Funções como valores

LJS-03.03 - Estudo - Escopo, bindings e closures

LJS-03.03 - Prática - Escopo, bindings e closures

LJS-03.04 - Estudo - Chamadas e execução síncrona
LJS-03.04 - Prática - Chamadas e execução síncrona

LJS-03 - Checkpoint - Funções, escopo e abstração

LJS-04 - Modelagem de dados, identidade e mutabilidade

LJS-04.01 - Estudo - Sequências e registros
LJS-04.01 - Prática - Sequências e registros

LJS-04.02 - Estudo - Identidade, compartilhamento e cópia
LJS-04.02 - Prática - Identidade, compartilhamento e cópia

LJS-04 - Checkpoint - Modelagem de dados, identidade e mutabilidade

LJS-05 - Processamento de coleções e raciocínio algorítmico

LJS-05.01 - Estudo - Padrões de processamento
LJS-05.01 - Prática - Padrões de processamento

LJS-05.02 - Estudo - Expressando padrões em JavaScript
LJS-05.02 - Prática - Expressando padrões em JavaScript

LJS-05.03 - Estudo - Ordenação e custo intuitivo
LJS-05.03 - Prática - Ordenação e custo intuitivo

LJS-05 - Checkpoint - Processamento de coleções e raciocínio algorítmico

LJS-06 - Objetos, protótipos e classes

LJS-06.01 - Estudo - Propriedades e cadeia de protótipos
LJS-06.01 - Prática - Propriedades e cadeia de protótipos

LJS-06.02 - Estudo - Métodos, this e construção
LJS-06.02 - Prática - Métodos, this e construção

LJS-06.03 - Estudo - Classes sobre o modelo prototípico
LJS-06.03 - Prática - Classes sobre o modelo prototípico

LJS-06 - Checkpoint - Objetos, protótipos e classes

LJS-07 - Módulos e falhas

LJS-07.01 - Estudo - Módulos ECMAScript
LJS-07.01 - Prática - Módulos ECMAScript

LJS-07.02 - Estudo - Erros e exceções
LJS-07.02 - Prática - Erros e exceções

LJS-07 - Checkpoint - Módulos e falhas

LJS-08 - Execução assíncrona e ambientes JavaScript

LJS-08.01 - Estudo - Linguagem, engine, runtime e host

LJS-08.01 - Prática - Linguagem, engine, runtime e host

LJS-08.02 - Estudo - Promises

LJS-08.02 - Prática - Promises

LJS-08.03 - Estudo - async e await

LJS-08.03 - Prática - async e await

LJS-08.04 - Estudo - Jobs, tasks e microtasks

LJS-08.04 - Prática - Jobs, tasks e microtasks

LJS-08 - Checkpoint - Execução assíncrona e ambientes JavaScript

LJS - Checkpoint Final

Competências ao concluir o Lab

1. **Analisar** um problema em termos de entradas, saídas, restrições, estados e transformações.
2. **Decompor** soluções em algoritmos e funções com responsabilidades claras.
3. **Prever** a execução de expressões, decisões, loops, chamadas e operações assíncronas.
4. **Representar** estado corretamente usando valores, bindings, arrays e objetos.
5. **Modelar** dados considerando identidade, compartilhamento e mutabilidade.
6. **Transformar** coleções por padrões de percurso, busca, seleção, transformação e agregação.
7. **Abstrair** comportamento usando funções, callbacks, closures e funções de ordem superior.
8. **Explicar** escopo, call stack, protótipos, **this**, classes, módulos e Promises sem recorrer a modelos tecnicamente enganosos.
9. **Organizar** programas em módulos e raciocinar sobre propagação de erros.
10. **Distinguir** linguagem ECMAScript, engine, runtime, host e APIs externas, criando base suficiente para avançar posteriormente para browser, Node.js e outras plataformas.

Aprofundamentos posteriores

- **Algoritmos e estruturas de dados:** recursão sistemática, stacks, queues, linked lists, trees, graphs, heaps, hash tables, algoritmos clássicos, busca binária e implementação de algoritmos de ordenação.
- **Análise algorítmica:** Big-O, Θ e Ω , análise formal de tempo e espaço, estratégias algorítmicas e provas de correção.
- **Coleções adicionais:** Map, Set, WeakMap, WeakSet e critérios avançados de escolha de estruturas.
- **Sistema de tipos e valores:** BigInt, Symbol e detalhes numéricos mais profundos.
- **Objetos avançados:** getters/setters, property descriptors, propriedades privadas, membros **static**, extends, super, explicit binding avançado de **this**, Proxy e Reflect.
- **Protocolos da linguagem:** iterables, iterators, generators, async iterators e generators assíncronos.
- **Processamento especializado:** expressões regulares, typed arrays e dados binários.
- **Assincronismo avançado:** demais combinadores de Promises, estratégias de cancelamento fornecidas por APIs, limitação de concorrência e workers.
- **Memória:** garbage collection, reachability, WeakRef e modelos de memória avançados.
- **Legado e interoperabilidade:** var, padrões históricos de funções construtoras, callbacks profundamente aninhados e CommonJS.
- **Ambientes e engenharia:** DOM, Web APIs, Node.js, package management, testes, debugging sistemático, linting, TypeScript e frameworks.